

AI Recommendation

Từ tín hiệu hành vi đến một sản phẩm được xếp hạng #1

Deep dive vào candidate generation, embedding, pgvector, hybrid scoring, merge, exclusion, cache và feedback loop trong chính dự án.

384D

Embedding

0.75 / 0.25

Hybrid

4 feeds

Serving

Nội dung bám sát: Node.js recommendation repository + FastAPI AI service + MySQL/Redis/RabbitMQ + PostgreSQL/pgvector

Câu hỏi trung tâm: vì sao sản phẩm này được gợi ý?

Không bắt đầu từ endpoint; bắt đầu từ bằng chứng và phép tính.

1 • Bằng chứng người dùng

Người dùng xem váy ren, thêm váy chữ A vào giỏ, tìm “váy trắng dự tiệc”. Mỗi hành vi có trọng số và cửa sổ thời gian khác nhau.

2 • Bằng chứng sản phẩm

Tên, mô tả, danh mục và tags được biến thành vector 384 chiều. Khoảng cách cosine đo mức gần về ngữ nghĩa.

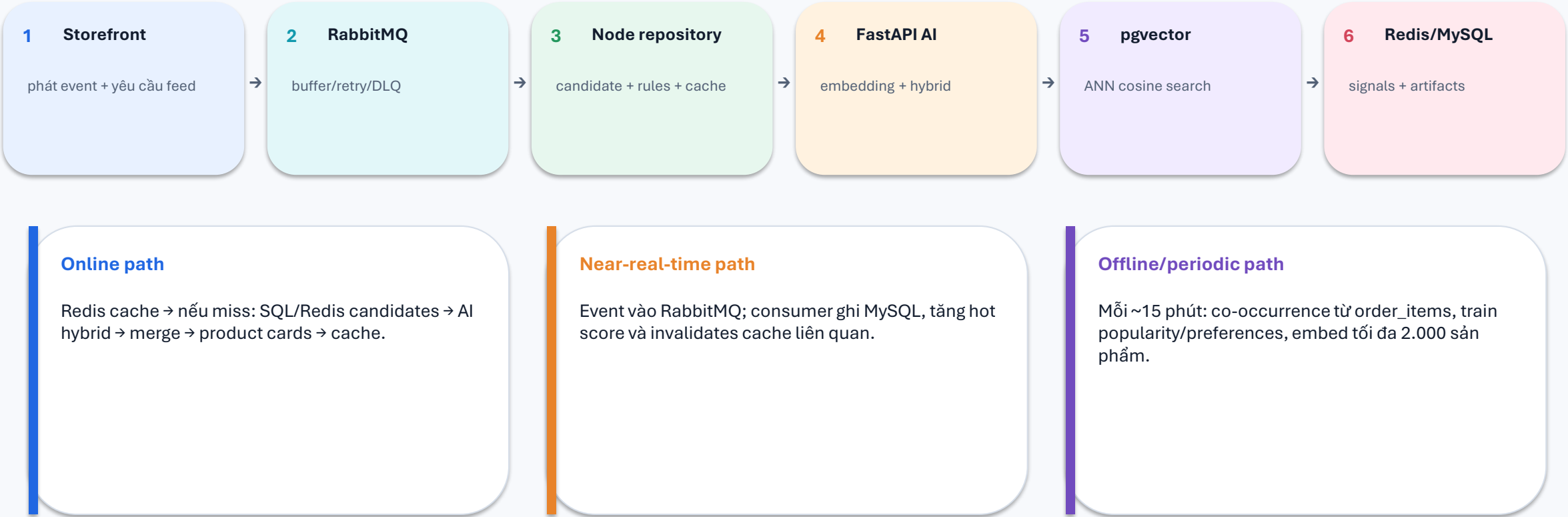
3 • Quyết định xếp hạng

Nhiều nguồn sinh candidate; hệ thống loại sản phẩm đã mua/đang ở giỏ, hợp nhất điểm, sắp xếp và cache kết quả.

Output = candidate hợp lệ có bằng chứng mạnh nhất tại placement hiện tại

Kiến trúc triển khai thực tế

Hai lớp ranking: orchestration trong Node.js và semantic retrieval trong AI service.



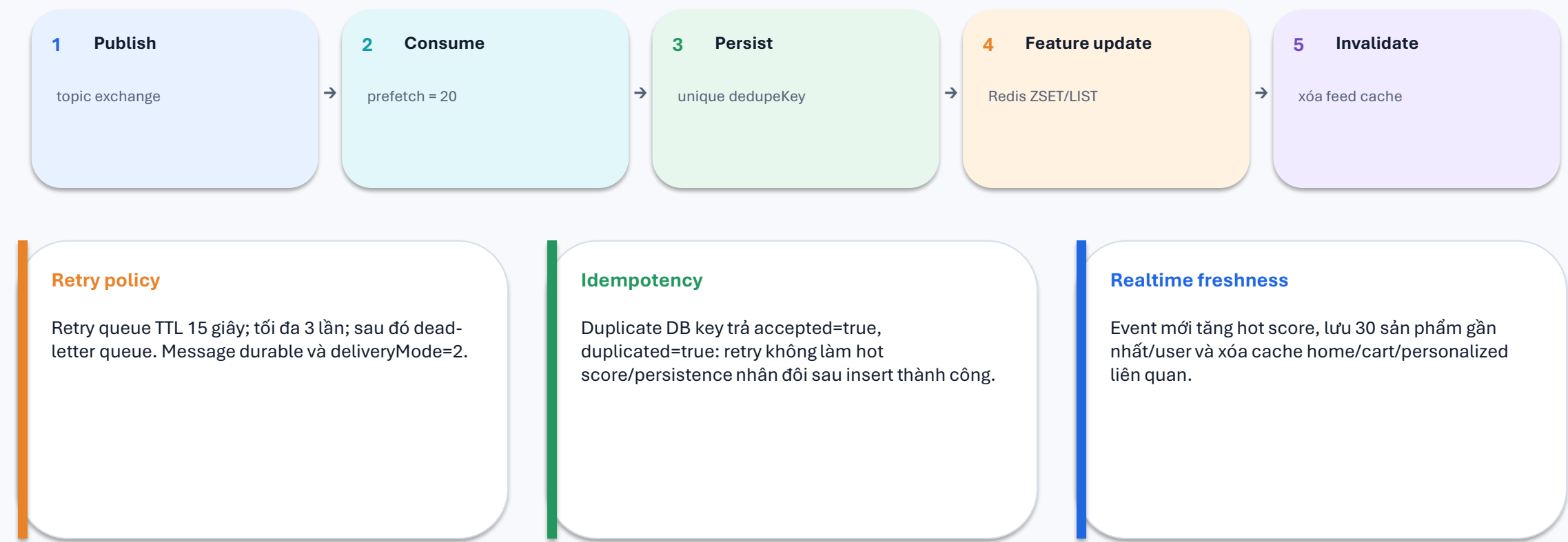
Dữ liệu đầu vào: một event không chỉ là một click

Schema RecommendationEvent giữ identity, context, placement và chống trùng.

Trường	Ý nghĩa kỹ thuật	Ví dụ
eventType	tín hiệu implicit/explicit	VIEW_PRODUCT
userId	cá nhân hóa đăng nhập	user-123
sessionId	cold-start/anonymous context	sess-a9
productId	đối tượng hành vi	dress-rumy
searchQuery	ý định bằng ngôn ngữ	váy trắng
placement	đo hiệu quả vị trí	home_recommendations
dedupeKey	idempotency unique	view:....timestamp
occurredAt	recency window	2026-07-12

Ingestion đáng tin cậy: event đi qua RabbitMQ

Tách latency của UI khỏi persistence và feature update.



Trọng số hành vi: giá trị kinh doanh được mã hóa thành số

Các pipeline dùng trọng số hơi khác nhau theo mục tiêu feed.

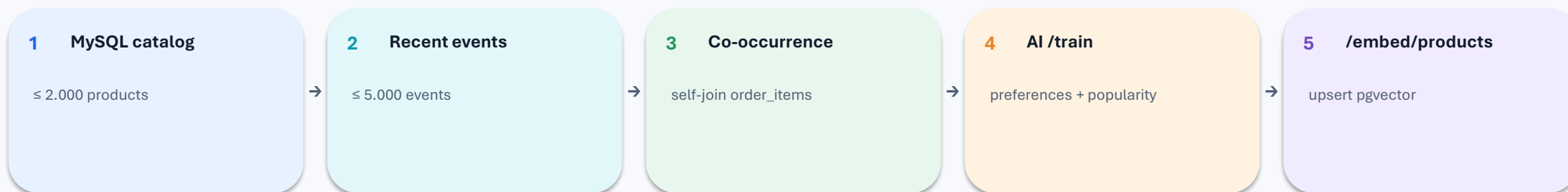
Event	Hot Redis	Trending 30d	Personalized 45d	AI train
PURCHASE	5	5	4×1.35	4
ADD_TO_CART	3	3	3×1.35	3
FAVORITE	2	2	2.5×1.35	2
VIEW	1	1	1×1.35	1
REC_CLICK	1.5	—	0.5×1.35	1
IMPRESSION	0	—	0.5×1.35	1

Ý nghĩa

Purchase mạnh hơn view; personalized còn boost 35% tín hiệu thuộc user. Đây là heuristic, chưa phải trọng số được học từ dữ liệu.

Offline refresh: biến lịch sử thành artifacts phục vụ nhanh

Scheduler chạy ngay lúc bootstrap và lặp theo RECOMMENDATION_REFRESH_INTERVAL_MS (mặc định 15 phút).



Co-occurrence artifact

$\text{score}(A,B)$ = số đơn chứa đồng thời A và B. Lưu `product_similarities` với `algorithm='cooccurrence', score, rank, metadata refreshedAt`.

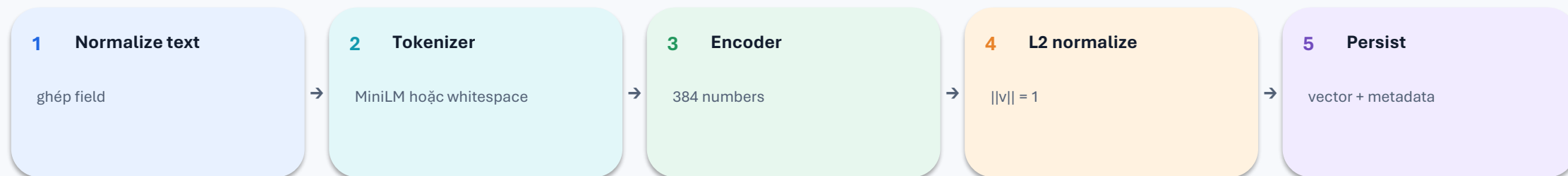
Semantic artifact

`document` = title + description + category + tags/attributes → normalized embedding 384D → PostgreSQL vector index.

Tạo document cho một sản phẩm

Embedding tốt hay xấu phụ thuộc trực tiếp vào chuỗi đầu vào.

"Váy ren dáng A Rummy" + "cổ bèo, dự tiệc..." + "Váy nữ" + "color trắng occasion party material lace"



Điểm mạnh

Tên + mô tả + category + tags giúp semantic retrieval hiểu “váy tiệc” gần “đầm dự sự kiện”.

Giới hạn hiện tại

Variant attributes chi tiết chưa được sync đầy đủ; payload Node chủ yếu đưa tags vào attributes.

Encoder: MiniLM nếu tải được, hashing nếu không

Hệ thống có graceful fallback nhưng chất lượng hai chế độ rất khác nhau.

all-MiniLM-L6-v2

SentenceTransformer encode(text, normalize_embeddings=True).
Học semantic từ corpus lớn; từ đồng nghĩa có thể gần nhau dù token khác nhau. Output 384 chiều.

hashing-baseline

Tách theo whitespace → hash(token) mod 384 → cộng tần suất → L2 normalize. Nhanh và offline, nhưng collision và không hiểu đồng nghĩa/ngữ cảnh.

Rủi ro vận hành: health.model_loaded=false không làm service chết, nhưng ranking semantic có thể giảm mạnh mà người dùng chỉ thấy chất lượng kém.

Cosine similarity trong pgvector

Tìm vector gần context nhất bằng toán tử <=>.

$$\cos(q,p) = (q \cdot p) / (\|q\| \|p\|)$$

Vì vector đã normalize L2:

$$\cos(q,p) \approx q \cdot p$$

SQL score = 1 - (embedding <=> query_vector)

0.92

Rất gần về nội dung/ngữ nghĩa

0.65

Có một phần thuộc tính/danh mục chung

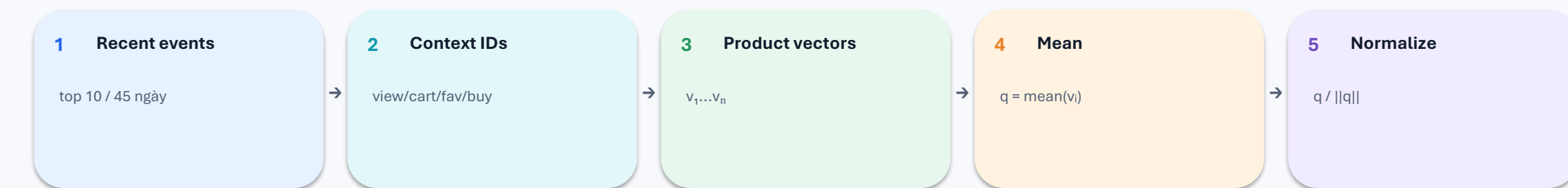
0.10

Hầu như không liên quan

IVFFlat + vector_cosine_ops: giảm phạm vi tìm kiếm so với full scan

Context vector: hệ thống hiểu 'gu hiện tại' như thế nào?

Không embed user trực tiếp trong request; lấy trung bình vector các sản phẩm context.



$$q_user = \text{normalize}((v_váy-ren + v_váy-chữ-A + v_áo-cổ-bèo) / 3)$$

- Nếu user đăng nhập: AI còn nối tối đa 8 sản phẩm preference phổ biến vào context.
- user_profile tuổi/chiều cao/cân nặng được gửi sang AI nhưng hiện chưa tham gia công thức recommend().
- Nếu không có context vector: AI vector_hits rỗng, chỉ còn popularity fallback.

Công thức hybrid trong AI service

Semantic relevance chiếm 75%; demand evidence chiếm 25%.

$$\text{final_AI}(p) = 0.75 \times \text{cosine}(q,p) + 0.25 \times \log(1 + \text{popularity}(p))$$

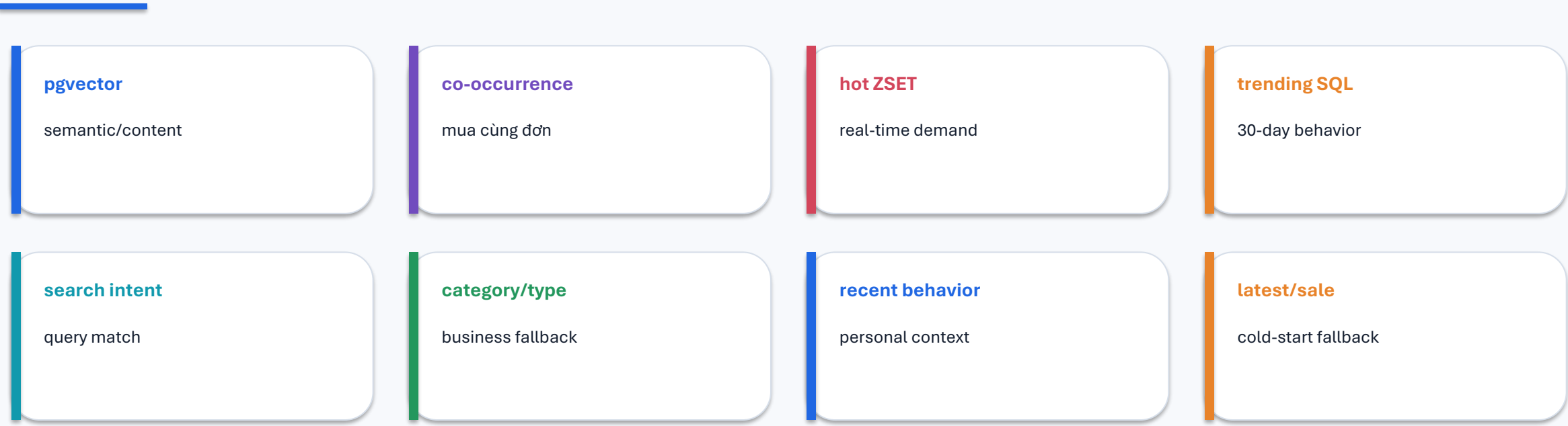
Candidate	Cosine	Raw popularity	log1p(pop)	Final AI
Đầm dự tiệc B	0.91	4	1.609	1.085
Váy basic C	0.72	20	3.045	1.301
Áo ren D	0.83	1	0.693	0.796

Kết luận ví dụ

C vượt B dù semantic thấp hơn, vì popularity chưa được normalize và log1p có thể > 1. Đây là chủ đề calibration quan trọng.

Candidate generation: không có một 'model duy nhất'

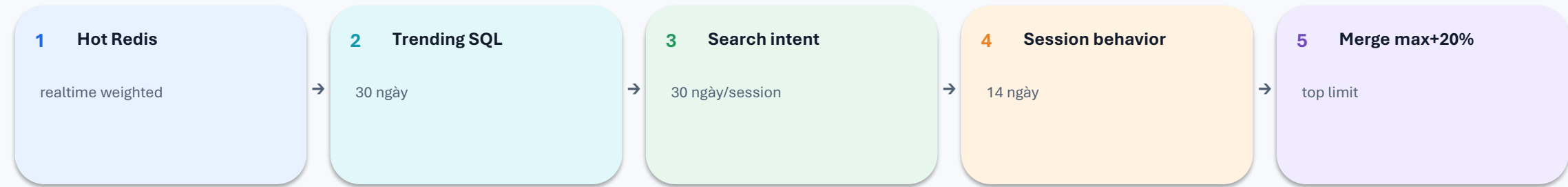
Mỗi nguồn trả productId + score + reason + source.



Candidate generation ưu tiên recall: lấy limit × 2/3/4 rồi mới merge, exclude và cắt top-K.

Feed Home: xu hướng + phiên anonymous + ý định tìm kiếm

Mục tiêu là relevance tức thời ngay cả khi chưa đăng nhập.



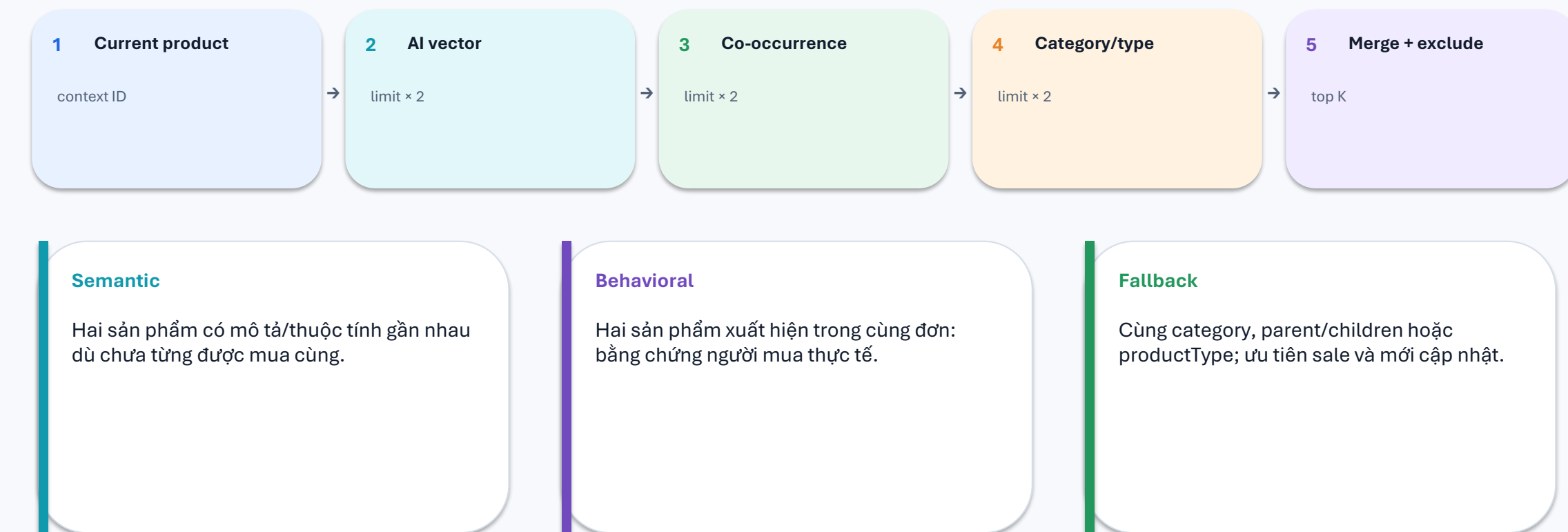
Session event	Weight
ADD_TO_CART	3
FAVORITE_PRODUCT	2.5
VIEW_PRODUCT	1
khác	0.5

Cache & fallback

Cache theo session + limit, TTL 900s. Nếu không có context: sản phẩm có ảnh, ưu tiên isSale rồi createdAt mới nhất.

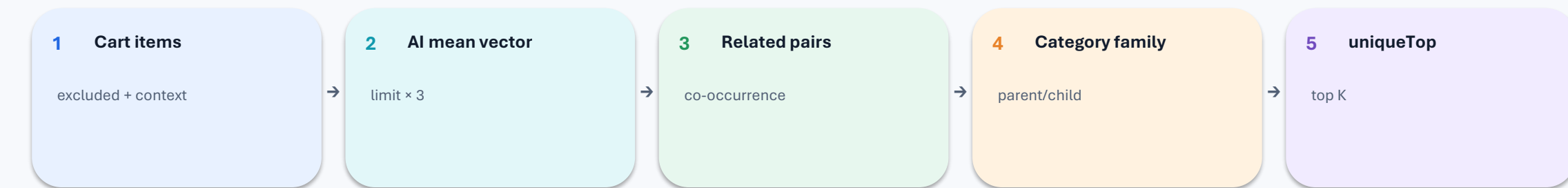
Feed Product Detail: 'sản phẩm tương tự' được tạo thế nào?

Context là đúng một product đang xem; output luôn loại chính product đó.



Feed Cart: cross-sell từ nhiều sản phẩm context

Không gợi ý lại bất kỳ product nào đang nằm trong giỏ.

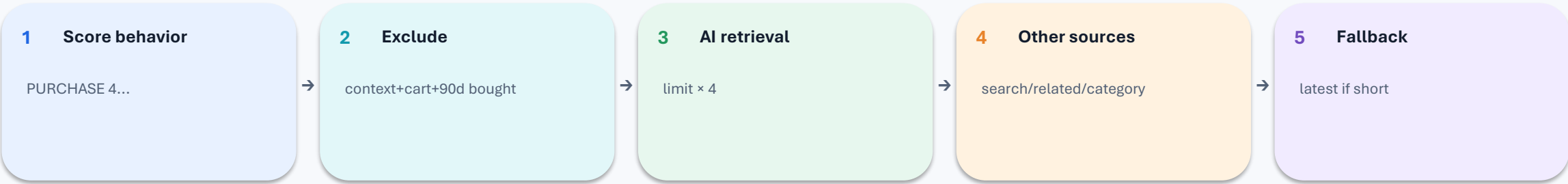


Ví dụ: giỏ = {váy ren, túi trắng} → context mean mang cả occasion + style → candidate giày cao gót có thể gần cả hai.

- AI reason: “phù hợp với các sản phẩm trong giỏ”.
- Co-occurrence cung cấp cross-sell dựa trên đơn thật.
- TTL 600 giây; ADD/REMOVE event invalidates cart cache.

Feed Personalized: pipeline đầy đủ nhất

Kết hợp behavior 45 ngày, search intent, AI, co-occurrence và category.



Own signals

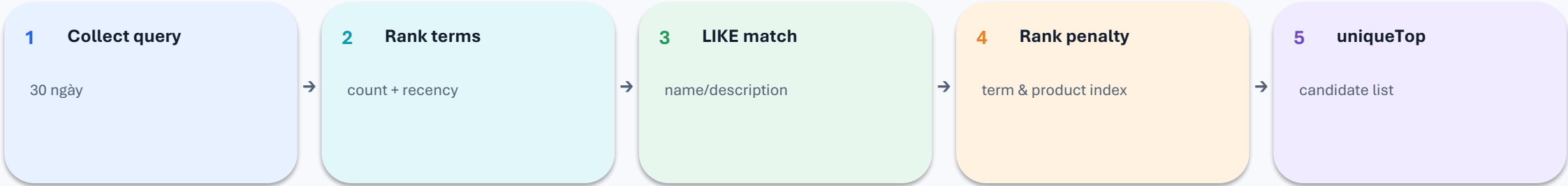
recent event score × 1.2 được đưa lại làm candidate, nhưng contextIds đồng thời nằm trong exclusion nên thực tế các own candidates này thường bị loại ở uniqueTop.

Exclusion

Không gọi lại sản phẩm context, sản phẩm đang trong cart và mọi sản phẩm đã mua trong 90 ngày—giảm lặp nhưng có thể bỏ lỡ replenishment.

Search intent: chuyển câu người dùng thành candidate

Hiện dùng lexical SQL LIKE, chưa embed query.



$$\text{score} = \max(0.5, \text{query_count} \times 3 - \text{termIndex} \times 0.75 - \text{productIndex} \times 0.25)$$

Tốt

Phản ứng nhanh với “váy trắng”, dễ giải thích.

Chưa tốt

Không hiểu synonym/typo; query score có thang khác cosine.

Co-occurrence: học quan hệ từ giỏ hàng đã mua

Một dạng item-item collaborative signal đơn giản, minh bạch.

SQL self-join

```
order_items oi1
JOIN order_items oi2
  ON same_order_id
AND product_id khác nhau

score(A,B) = COUNT(*)
```

A + B cùng 12 đơn

score(A,B)=12

A + C cùng 4 đơn

score(A,C)=4

Vấn đề popularity bias

Không normalize theo độ phổ biến A/B; bestseller dễ liên quan với mọi thứ.

Merge scores trong Node: max thắng, nguồn phụ cộng 20%

Đây là bước cuối quyết định candidate nào đứng trên candidate nào.

Nếu product xuất hiện nhiều lần: giữ source có score lớn nhất; mỗi score nhỏ hơn đến sau cộng thêm $0.2 \times \text{score}$ đó.

Product	pgvector	cooccur	category	Kết quả merge
B	1.085	0.80	—	$1.085 + .2 \times .80 = 1.245$
C	1.301	—	8.0	$8.0 + .2 \times 1.301 = 8.260$
D	0.796	1.20	7.0	$7.0 + .2 \times 1.2 + .2 \times .796 = 7.399$

Technical debt

Score cosine (~0–1), co-occurrence (count), fallback rank (limit...1), search (count×3) không cùng thang; category fallback có thể lấn át AI.

Bộ lọc và guardrail trước khi hiển thị

Ranking tốt nhưng candidate không hợp lệ vẫn phải bị loại.

Context

không gợi lại item đang xem

Cart

không gợi item trong giỏ

Purchased 90d

giảm lặp personalized

Soft delete

product isDeleted=false

Card lookup

missing product bị drop

Unique

mỗi product một lần

Khoảng trống hiện tại: getProductCards chưa lọc Product.status ACTIVE, variant stock/status hay stockAvailable > 0.

Worked example #1: AI chọn candidate nào?

Context: user vừa xem váy ren trắng và váy chữ A; candidate là B/C/D.

Candidate	Cosine	Events pop	log1p	0.75×cos	0.25×log	AI score
B Đầm dự tiệc	0.91	4	1.609	0.683	0.402	1.085
C Váy basic	0.72	20	3.045	0.540	0.761	1.301
D Áo ren	0.83	1	0.693	0.623	0.173	0.796

#1 C • 1.301

Demand thắng semantic

#2 B • 1.085

Semantic cao nhất

#3 D • 0.796

Ít popularity

Nếu business muốn semantic chi phối thật sự, popularity cần normalize (min-max/percentile) trước khi nhân 0.25.

Worked example #2: sau merge, thứ hạng có thể đảo

AI score không phải final score của Node repository.

Product	Nguồn mạnh nhất	Nguồn phụ	Final Node	Rank
C Váy basic	category=8.0	AI=1.301	8.260	#1
D Áo ren	category=7.0	co=1.2; AI=.796	7.399	#2
B đầm tiệc	AI=1.085	co=.80	1.245	#3

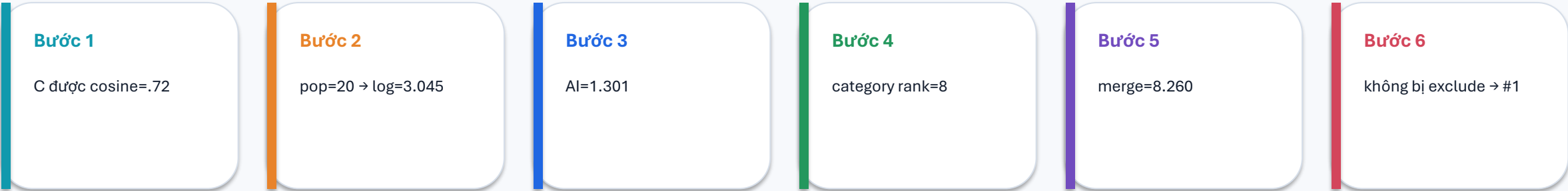
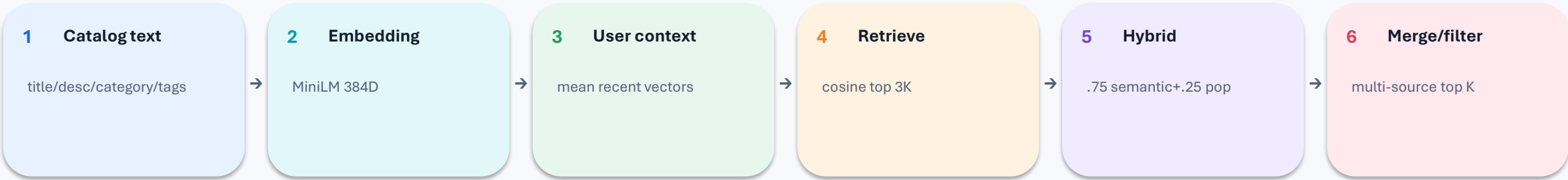


Thông điệp bảo vệ

“AI retrieval tạo candidate tốt; orchestration + heuristic calibration mới quyết định sản phẩm cuối cùng.”

End-to-end: một sản phẩm đi từ catalog đến vị trí #1

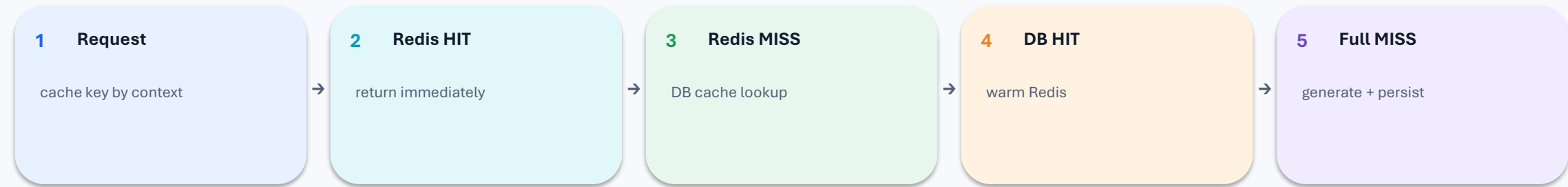
Chuỗi quyết định đầy đủ, không bỏ qua tầng dữ liệu hay serving.



Sản phẩm được gợi ý không vì “API trả về”, mà vì sống sót qua 6 phép biến đổi và guardrail.

Cache hai tầng: tốc độ mà vẫn có khả năng phục hồi

Redis là fast path; MySQL RecommendationCache là durable fallback.



Feed	Cache key	TTL
Home	home:v2:{session}:{limit}	900s
Product	product:{product}:{limit}	1800s
Cart	cart:{user}:{limit}	600s
Personalized	personalized:v4:{user}:{session}:{limit}	900s

Cache invalidation: freshness đến từ event

Không chờ TTL nếu hành vi vừa thay đổi.

Event context	Patterns bị xóa
sessionId	home:{session}:*, home:v2:{session}:*, personalized*:{session}:*
userId	cart:{user}:*, personalized v1/v2/v3/v4 của user

Ưu điểm

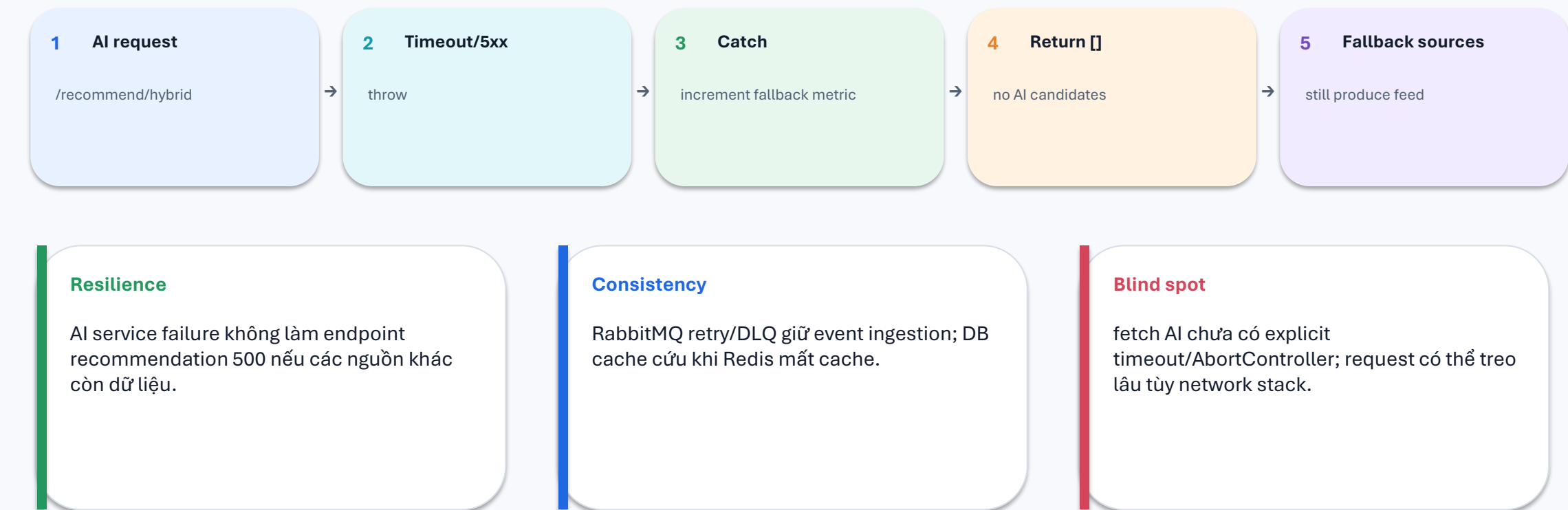
ADD_TO_CART làm feed cart/personalized thay đổi ngay ở request kế tiếp.

Rủi ro

Dùng Redis KEYS(pattern) có thể block khi keyspace lớn; production nên chuyển SCAN hoặc versioned keys.

Fault tolerance: AI hỏng thì feed không nhất thiết hỏng

Node repository bắt exception và tiếp tục bằng co-occurrence/category/catalog.



Observability: biết hệ thống nhanh hay chỉ 'có chạy'

Prometheus metrics được đo ở event, cache, feed và AI.

Metric	Labels	Câu hỏi trả lời
recommendation_events_total	event_type, source	Có nhận đúng hành vi?
cache_hits_total	feed	Cache hiệu quả bao nhiêu?
generation_latency_ms	feed	P95 feed nào chậm?
ai_latency_ms	operation	Train/embed/recommend chậm?
ai_fallback_total	feed	AI hỏng hoặc suy giảm?

Nên bổ sung: candidate count/source, zero-result rate, score distribution, CTR/CVR theo model version

Đánh giá chất lượng: offline metric chưa đủ

Recommendation phải đo cả relevance, diversity và tác động kinh doanh.

Offline ranking

Recall@K: item mua có nằm trong candidate?
NDCG@K: item đúng đứng cao?
MRR: vị trí item đầu tiên đúng.
Coverage: % catalog được expose.

Online behavior

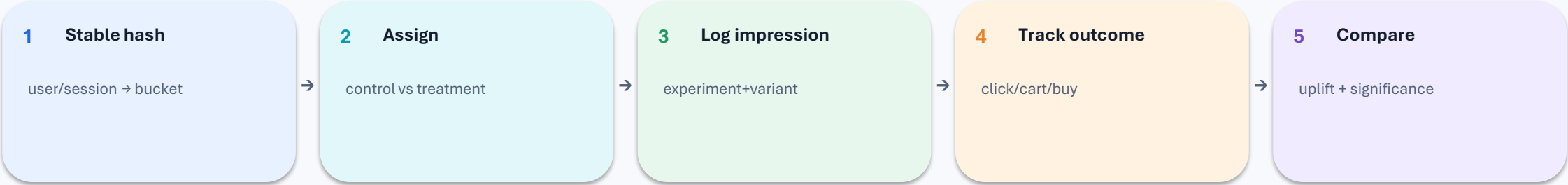
Impression → click CTR.
Click → add-to-cart.
Recommendation-assisted purchase.
Revenue/session và conversion uplift.

Guardrail

Latency P95/P99.
Out-of-stock exposure.
Diversity category/brand.
Popularity concentration và novelty.

A/B testing đúng cách

Schema RecommendationExperiment đã có, nhưng serving chưa gắn assignment/model variant.



Control

Heuristic hiện tại: unnormalized merge + 0.75/0.25 AI.

Treatment đề xuất

Normalize per-source → weighted RRF/learning-to-rank; diversity reranker; stock guardrail.

Technical debt nhìn thẳng vào code

Những điểm cần nói rõ để bài bảo vệ có chiều sâu và trung thực.

Score calibration

nguồn dùng thang điểm khác nhau

Body profile unused

được gửi nhưng AI không dùng

User embeddings unused

schema có, serving chưa đọc

Product status/stock

card filter chưa đủ guardrail

Hash reproducibility

Python hash đổi theo process seed

IVFFlat tuning

chưa thấy ANALYZE/lists/probes

Own signal conflict

recent items bị exclusion loại

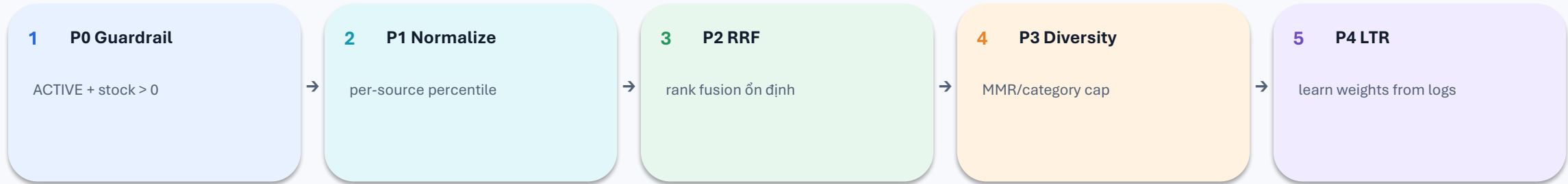
No query embedding

search chỉ LIKE

Điểm mạnh: hệ thống có đường tiến hóa rõ—không cần phá kiến trúc để thay heuristic bằng model tốt hơn.

Roadmap kỹ thuật: từ heuristic hybrid đến ranking học được

Ưu tiên theo impact/risk, không chạy theo độ phức tạp mô hình.



Giai đoạn	Thay đổi	Kết quả mong đợi
1–2 tuần	metrics + guardrail + SCAN cache	an toàn production
3–4 tuần	normalize + calibrated weights	ranking ổn định
5–8 tuần	A/B + MMR + query embedding	CTR/diversity tăng
>8 tuần	LightGBM/XGBoost LTR	trọng số học từ outcome

Kịch bản demo kỹ thuật

Chứng minh từng tầng bằng dữ liệu thay vì chỉ nhìn carousel.



1	Track VIEW/ADD	kiểm tra recommendation_events + Redis hot score
2	Refresh artifacts	kiểm tra co-occurrence + product embeddings
3	Query pgvector	so cosine của 3 candidate
4	Call feed	đọc score/reason/source
5	Trigger cache	request lần 2 và metric hit
6	New event	cache invalidated, ranking đổi

Demo tốt nhất kết thúc bằng việc giải thích vì sao rank thay đổi sau một ADD_TO_CART.

Kết luận: recommendation là một chuỗi quyết định có thể kiểm chứng

Từ product text và hành vi đến candidate cuối cùng đều có dữ liệu, công thức và fallback.

Retrieval

Embedding 384D + cosine/pgvector mở rộng semantic recall; co-occurrence và category cứu cold-start/AI failure.

Ranking

Hybrid 0.75/0.25 tạo AI score; Node merge max+20%, exclusion và fallback quyết định final rank.

Learning loop

Event → RabbitMQ → MySQL/Redis → refresh artifacts → cache invalidation → recommendation mới.

Một sản phẩm đứng #1 vì nó có bằng chứng phù hợp nhất sau toàn bộ pipeline—not because an API happened to return it.